Nonlinear Equations - Part 3
Polynomial Interpolation

Alexandra A. Gomes

April 12, 2026

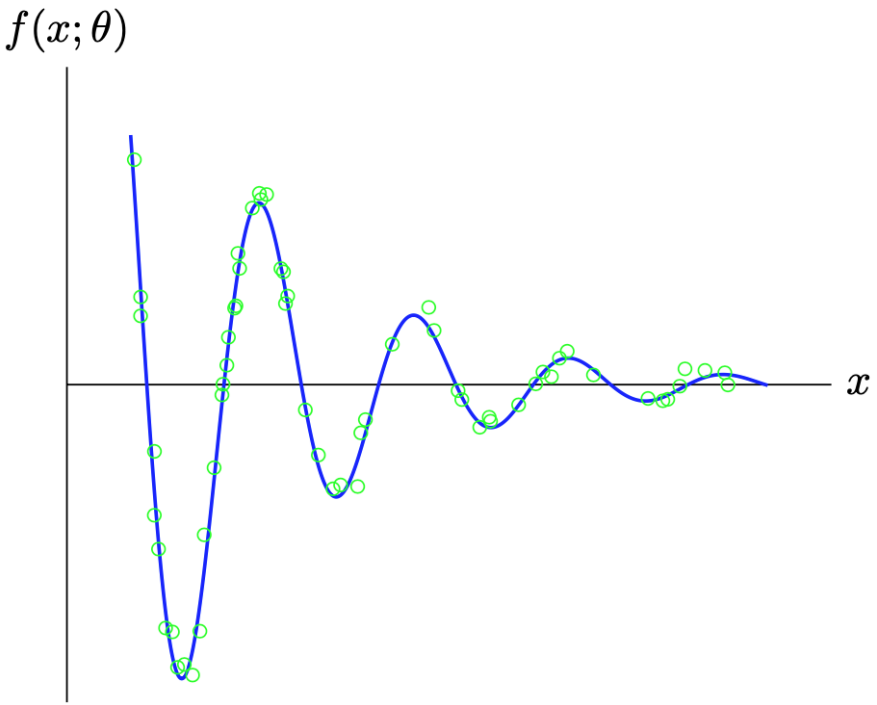
Nonlinear Data Fitting

- we are given a set of data: $x^{(1)}, \ldots x^{(N)}$, $y^{(1)}, \ldots, y^{(N)}$;
- we want to fit a model $y \approx f(x; \theta)$ to the given data, where the $p \times 1$ vector θ contains the model parameters;
- in linear data fitting, $f(x; \theta)$ is a linear function of the parameters,

$$f(x; \theta) = \theta_1 f_1(x) + \cdots + \theta_p f_p(x),$$

  and $f_1, \ldots, f_p$ are the basis functions;
- in nonlinear data fitting, the dependence of $f(x; \theta)$ on θ is not linear, so it does not have the simple form of a linear combination of p basis functions.

Example



$$f(x;\theta) = \theta_1 e^{\theta_2 x} \cos(\theta_3 x + \theta_4)$$

Nonlinear Least Squares Method

- suppose f is a real valued function;
- as in the linear case, find the vector parameter θ by minimizing the sum of the squares of the residuals

$$\sum_{i=1}^{N}(f(x^{(i)};\theta) - y^{(i)})^2;$$

- however, the gradient of the sum of the squares of the residuals is not linear in the components of θ;

Nonlinear Least Squares Method

$$\min_{\theta} \underbrace{\sum_{i=1}^{N} (f(x^{(i)};\theta) - y^{(i)})^2}_{\phi(\theta)} = \min_{\theta} \phi(\theta)$$

- solve $\nabla_\theta \phi = 0$ for the critical points;
- note that $\nabla_\theta \phi = 0$ is a system of nonlinear equations;
- use Newton's method to solve $\nabla_\theta \phi = 0$.

Gauss-Newton Method

- at each iteration k, form the linear approximation $\hat{f}$ of f at current iterate $\theta^{(k)}$, given by the Taylor approximation:

$$\hat{f}(\theta) = f(\theta^{(k)}) + \nabla_\theta f(\theta^{(k)})(\theta - \theta^{(k)});$$

- then, still at iteration k, choose $\theta^{(k+1)}$ to minimize the norm of the linearized residual, via linear least squares to update iterate $\theta^{(k)}$;
- the idea of linearizing and then solve the minimization problem using least squares is the basis of the Gauss-Newton Method.

Newton's Method for Unconstrained Optimization

$$\min_x \phi(x), \qquad x \in \mathbb{R}^n$$

- use Newton's method to solve $f(x) = \nabla\phi = 0$:

$$Df(x^{(k)})\Delta x = -f(x^{(k)}) \quad \Leftrightarrow \quad \nabla^2\phi(x^{(k)})\Delta x = -\nabla\phi(x^{(k)}),$$

  in which $\nabla^2\phi$ is the Hessian matrix of ϕ and

$$\Delta x = x^{(k+1)} - x^{(k)};$$

- issues:
  - existence and determination of $\nabla^2\phi$;
  - $\nabla^2\phi$ is not necessarily positive definite far from the minimizer;
  - solve a system of linear equations at each iteration.

Goal Polynomial Interpolation

We have a set of distinct points in $\mathbb{R}^2$ - $(x_i, f(x_i))$ - and we want to pass the graph of an algebraic polynomial through those points. That is, we search for an algebraic polynomial that agrees with, or **interpolates**, the values of f at those points.

An **algebraic polynomial** is a function of the form

$$p_n(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

in which, n is a positive integer and a_i are real constants.

Questions:

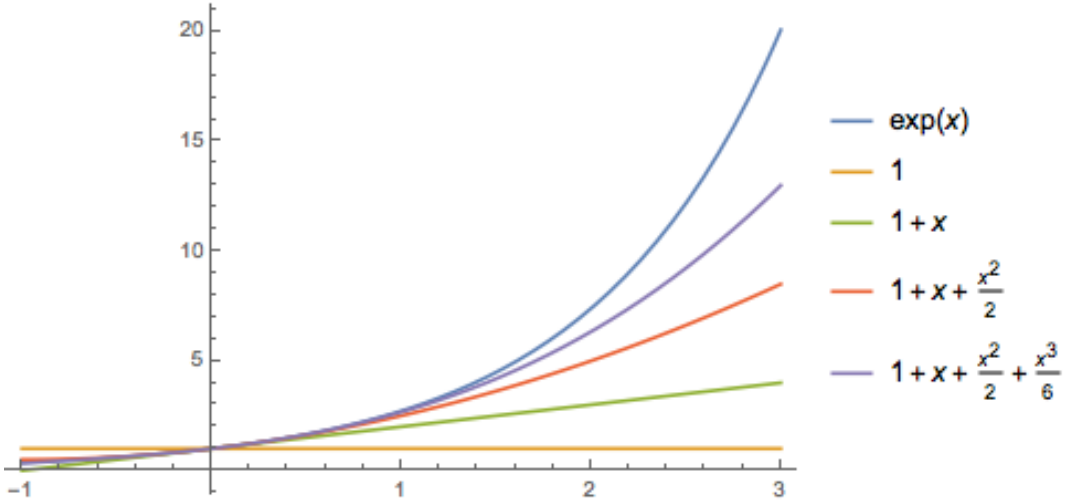
1. how do we construct the polynomial approximation?
2. what is the error in the approximation?

Why Polynomials?

- polynomials uniformly approximate continuous functions (Weierstrass Approximation Theorem);
- the derivative and the integral of a polynomial are easy to calculate and are also polynomials;
- Taylor polynomials concentrate accuracy near a specific point; for this reason, Taylor polynomials are not generally used in interpolation, in which accuracy is desired over an interval.

Taylor Polynomials as Interpolators: Example

1. e^x about $x_0 = 0$

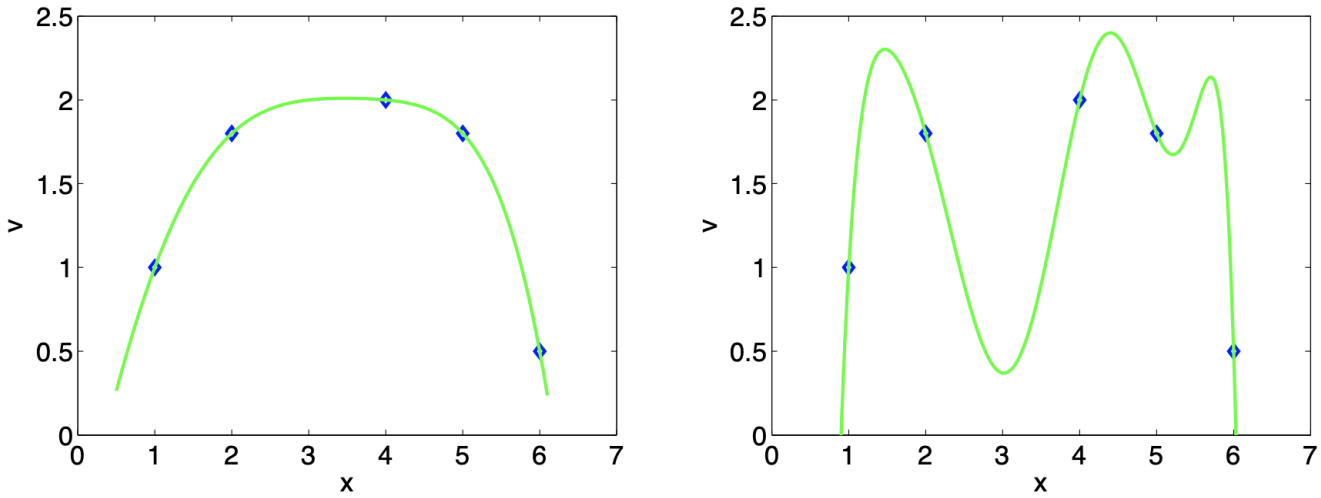


Motivation for Approximation

- we often get discrete data from sensors or computers, but we need to manipulate that information as if the data was continuous:
- we may need to plot, differentiate or integrate data trend;
- we may need the value of the original function at points other than the ones given.

Wish List

- we want a reasonable p_n:



  the interpolant on the left looks more reasonable than the one on the right;
- p_n should be inexpensive to construct and to evaluate;
- the construction of p_n should be stable numerically.

Questions

- is p_n unique for a set of $n+1$ points? yes!
- is there always an interpolant p_n? yes!
- how many times can a degree n polynomial oscillate?
- how do we choose n?
  1. if n in not large enough, approximation quality may be poor;
  2. if n is large, then we may get oscillations; can we reduce the magnitude of oscillations via a careful choice of x_i?

Interpolation Formulation

- we consider a linear combination of basis functions $f_i(x)$:

$$f(x) \approx p_n(x) = \theta_0 f_0(x) + \cdots + \theta_n f_n(x) = \sum_{j=0}^{n} \theta_j f_j(x);$$

- θ_i are the interpolation coefficients or interpolation weights, and can be computed from the $n+1$ interpolation conditions

$$\underbrace{\begin{bmatrix} f_0(x_0) & f_1(x_0) & \cdots & f_n(x_0) \\ f_0(x_1) & f_1(x_1) & \cdots & f_n(x_1) \\ \vdots & \vdots & & \vdots \\ f_0(x_n) & f_1(x_n) & \cdots & f_n(x_n) \end{bmatrix}}_{A} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Monomial Basis

- $n = 3$;
- let $f_j(x) = x^j, j = 0, \ldots n$: four basis functions: $\{1, x, x^2, x^3\}$;
- $\{(2, 14), (6, 24), (4, 25), (7, 15)\}$
- $$A = \begin{bmatrix} 1 & 2 & 4 & 8 \\ 1 & 6 & 36 & 216 \\ 1 & 4 & 16 & 64 \\ 1 & 7 & 49 & 343 \end{bmatrix}, \quad y = \begin{bmatrix} 14 \\ 24 \\ 25 \\ 15 \end{bmatrix}$$
- solve `theta = A \y` to find

$$\theta = [-0.267, 1.700, 2.767, 3.800]^T;$$

- then, the interpolant is $p_3(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$.

Monomial Basis: Basic Assessment

- simple to implement!
- however, A is a Vandermonde matrix, with a high condition number;
- polynomial construction cost is $O(2n^3/3)$ flop;
- evaluation of $p_n(x)$ requires $O(2n)$ flop with nested implementation;
- θ_i do not relate directly to $f(x_i)$;
- θ_i can change significantly if one data value is modified: the problem is ill conditioned;
- the monomial basis is not a good choice!
- conclusion: we need a better basis.

Existence of Unique Interpolating Polynomial

in previous example: A is a Vandermonde matrix: as long as the points x_i are distinct, A is nonsingular: there exists a unique solution for $A\theta = y$;

Theorem: given any real data points $(x_i, f(x_i)), i = 0, \ldots, n$, with x_i distinct, there exists a unique polynomial p of degree at most n such that

$$p(x_i) = f(x_i), \qquad i = 0, \ldots, n.$$

Lagrange Interpolating Polynomials: Idea

Given 2 points: (x_0, y_0), (x_1, y_1), we want a 1st order poly interpolating at the two points; define the functions

$$L_0(x) = \frac{x - x_1}{x_0 - x_1}, \qquad L_1(x) = \frac{x - x_0}{x_1 - x_0}$$

with the properties:

$$L_0(x_0) = 1; L_0(x_1) = 0; L_1(x_0) = 0; L_1(x_1) = 1.$$

Then, the Lagrange form of the linear interpolating poly through (x_0, y_0) and (x_1, y_1) is

$$P(x) = y_0 L_0(x) + y_1 L_1(x) = y_0 \frac{x - x_1}{x_0 - x_1} + y_1 \frac{x - x_0}{x_1 - x_0}.$$

Lagrange Interpolating Polynomials

Given $n+1$ points: $(x_0, f(x_0)), (x_1, f(x_1)), \ldots (x_n, f(x_n))$, define

$$L_{n,k}(x) = \frac{(x-x_0)(x-x_1)\ldots(x-x_{k-1})(x-x_{k+1})\ldots(x-x_n)}{(x_k-x_0)(x_k-x_1)\ldots(x_k-x_{k-1})(x_k-x_{k+1})\ldots(x_k-x_n)}$$

with the property:

$$L_{n,k}(x_i) = 1, i = k \qquad L_{n,k}(x_i) = 0, i \neq k.$$

$L_{n,k}(x_i) \rightarrow$ **nth order Lagrange Interpolating Polynomial**
Then,

$$p_n(x) = \sum_{i=0}^{n} f(x_i) L_{n,i}(x) = \sum_{i=0}^{n} f(x_i) \prod_{j=0, j\neq i}^{n} \frac{x-x_j}{x_i-x_j}.$$

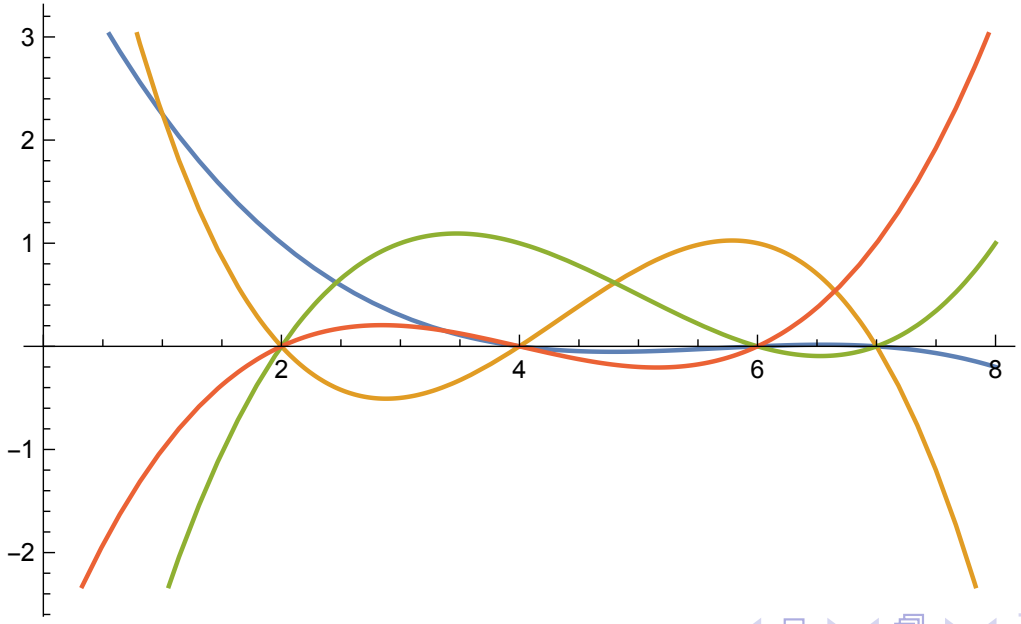
Lagrange Interpolating Polynomial: Properties

1. $L_k(x_i) = 1, i = k \qquad L_k(x_i) = 0, i \neq k;$
2. L_k has n zeros and $n-1$ extrema, alternating minima and maxima;
3. $L_k(x_k) = 1$ is not necessary an extrema;
4. the matrix of the system is now the identity matrix.

Example

- same data: $\{(2,14),(6,24),(4,25),(7,15)\}$;
- four Lagrange interpolating polynomials:

$$L_0(x) = \frac{(x-6)(x-4)(x-7)}{(2-6)(2-4)(2-7)}, \quad L_1(x) = \frac{(x-2)(x-4)(x-7)}{(6-2)(6-4)(6-7)},$$
$$L_2(x) = \frac{(x-2)(x-6)(x-7)}{(4-2)(4-6)(4-7)}, \quad L_3(x) = \frac{(x-2)(x-6)(x-4)}{(7-2)(7-6)(7-4)};$$



Example

- construction:

$$p_3(x) = 14\frac{(x-6)(x-4)(x-7)}{(2-6)(2-4)(2-7)} + 24\frac{(x-2)(x-4)(x-7)}{(6-2)(6-4)(6-7)}$$
$$+ 25\frac{(x-2)(x-6)(x-7)}{(4-2)(4-6)(4-7)} + 15\frac{(x-2)(x-6)(x-4)}{(7-2)(7-6)(7-4)}.$$

- naive evaluation: $O(n^2)$ flops per x;
- can we do better? yes!

Construction: Ordering Operations

- during construction use *barycentric weights*:

$$w_0 = \frac{1}{(2-6)(2-4)(2-7)}, \quad w_1 = \frac{1}{(6-2)(6-4)(6-7)}$$
$$w_2 = \frac{1}{(4-2)(4-6)(4-7)}, \quad w_3 = \frac{1}{(7-2)(7-6)(7-4)}$$

- during evaluation, at x define:

$$L(x) = (x-2)(x-6)(x-4)(x-7);$$
$$p_3(x) = L(x)\left[\frac{14w_0}{x-2} + \frac{24w_1}{x-6} + \frac{25w_2}{x-4} + \frac{15w_3}{x-7}\right]$$

Construction and Evaluation: Ordering Operations

- in general:

$$w_j = \frac{1}{\prod_{i \neq j}(x_j - x_i)}; L(x) = \prod_{i=0}^{n}(x - x_i),$$

$$p(x) = L(x) \sum_{j=0}^{n} \frac{f(x_j) w_j}{x - x_j};$$

- construction is $O(n^2)$;
- evaluation is $O(5n)$;

Matlab Implementation

```
function yt = lagrange_eval(w, x, y, t)
% function to evaluate a polynomial interpolant
% of the points [x, y] at the abcissa t;
% w are the barycentric weights
n = length(x);
L = prod(t - x);
yt = 0;
for j = 0:n
        yt = yt + y(j) * w(j) / (t-x(j));
end
yt = L * yt;
```

Matlab Implementation

```
% if t is exactly one of the x(j) points,
% the formula fails and produces NaN.
% Check for this condition and return y(j)
for j = 0:n
        if t == x(j)
                yt = y(j);
        end
end
```

Lagrange Polynomial Interpolation: Assessment

- not as simple as monomial basis;
- A is the identity: coefficients are immediately obtained;
- stable for large n and for spread apart nodes;
- construction cost is $O(n^2)$ flop;
- evaluation cost is $O(n)$ flop per x;
- coefficients θ_i are indicative of $f(x)$;
- **the Lagrange interpolant is mathematically the same as the monomial interpolant**: the only difference is the choice of basis in the construction phase.

Interpolation Error

1. What is the error involved in approximating a function by an interpolating poly?
2. Is f well approximated at intermediate points?
3. Will more nodes increase the quality of the interpolating polynomial?
4. Can we choose the location of the nodes to increase the quality of the interpolating polynomial?

Example

What is the error in replacing $\sin x$ by its interpolating poly constructed from 10 equally spaced nodes in $[0, 1.6875]$?

First Interpolation Error Theorem

Let $x_0, \ldots x_n$ be $n+1$ **distinct** nodes in $[a, b]$, $f \in C^{n+1}[a, b]$, and p_n the interpolating polynomial of degree $\leq n$ of f at these nodes. Then, for each $x \in [a, b]$, there exists a number $\xi(x)$ between the nodes, that is, in (a, b) such that

$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)\ldots(x - x_n).$$

Equally Spaced Nodes: Upper Bound Lemma

If the nodes are equally spaced in $[a, b]$,

$$x_i = a + ih, \qquad h = \frac{b-a}{n},$$

then, for any $x \in [a, b]$,

$$\prod_{i=0}^{n} |x - x_i| \leq \frac{1}{4} h^{n+1} n!$$

Equally Spaced Nodes: Second Interpolation Error Theorem

Let $x_0, \ldots x_n$ be $n+1$ equally spaced nodes in $[a, b]$ and $f \in C^{n+1}[a, b]$, such that

$$|f^{(n+1)}(x)| \leq M.$$

Let p_n be the interpolating polynomial of degree $\leq n$ of f at these nodes. Then, for each $x \in [a, b]$,

$$|f(x) - p_n(x)| \leq \frac{M}{4(n+1)} h^{n+1}.$$

Equally Spaced Nodes: Interpolation Error Theorem

the interpolation error, $e_n(x) = f(x) - p_n(x)$, verifies

$$|e_n(x)| \leq \frac{M}{4\,(n+1)} h^{n+1}.$$

Example

Discuss the error if $\sin x$ is replaced by an interpolating poly constructed from 10 equally spaced nodes in $[0, 1.6875]$.

- $f(x) = \sin x$, $n + 1 = 10$, $a = 0$, $b = 1.6875$;
- $f^{(10)}(x) = -\sin x$, so $M = 1$;
- then,
$$|e_n(x)| \leq 1.34 \times 10^{-9}$$
- p_9 represents the sine function with an error of at most 2 units in the ninth decimal place;
- thus, p_9 approximates the sine function with at least 8 decimal digits of accuracy.

Practicalities

In general, we do not have f or its derivatives (in data fitting). Is there a way to get a measure of the interpolation error?

- calculate interpolating polynomials, p_{n-1}, p_n, p_{n+1} using different subsets of the same set of nodes and compare results;
- divide the data set into 2 sets: use one set to construct the interpolant and the other to evaluate its accuracy.

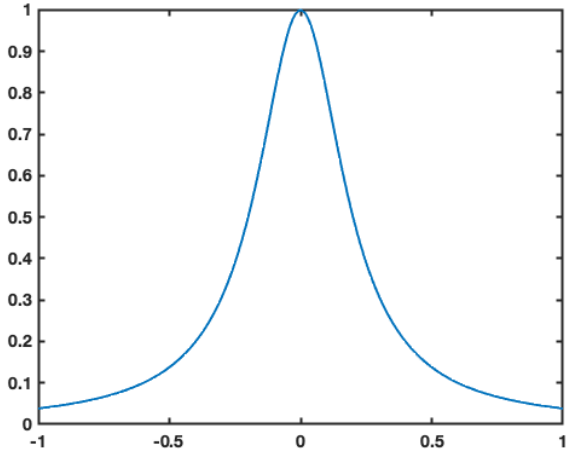
Interpolation Error

$$|e_n(x)| \leq \frac{M}{4\,(n+1)} h^{n+1}$$

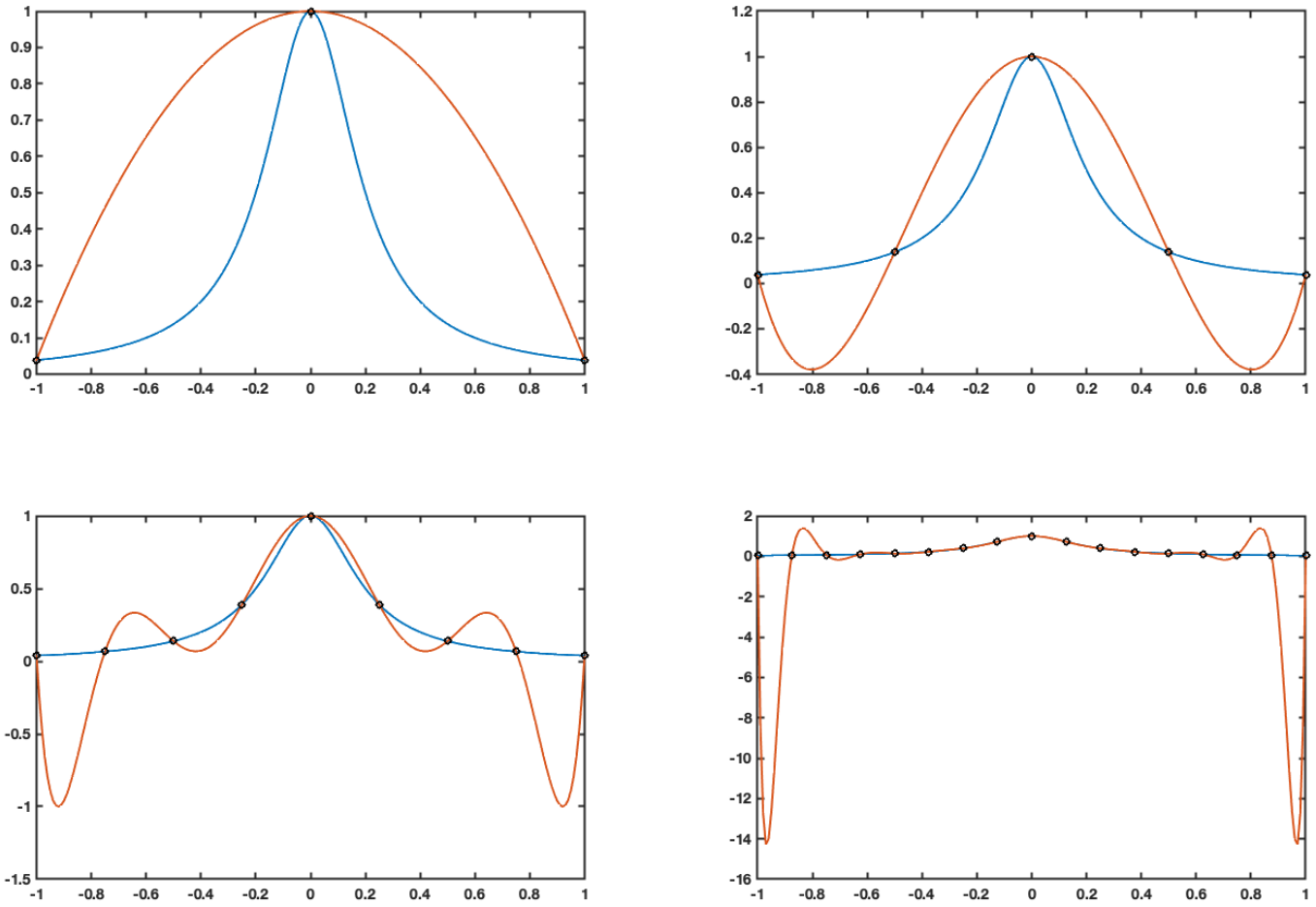
1. What is the error involved in approximating a function by an interpolating poly?
2. Is f well approximated at intermediate points?
3. Will more nodes increase the quality of the interpolating polynomial?
4. Can we choose the location of the nodes to increase the quality of the interpolating polynomial?

Runge Function

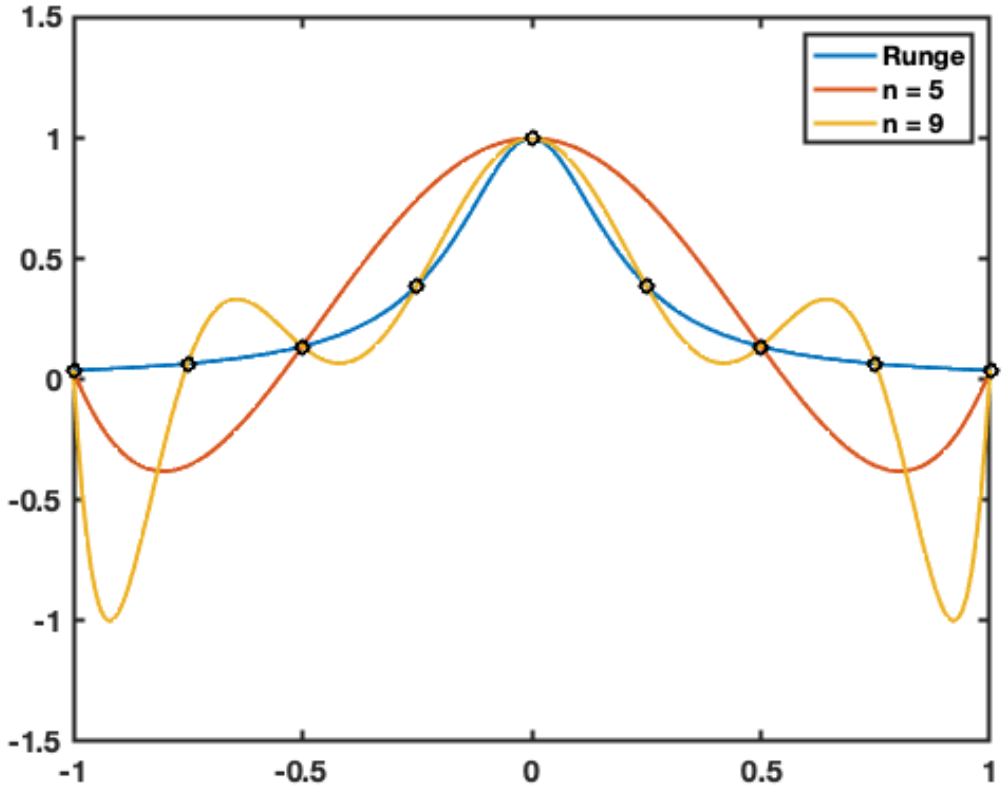
$$f(x) = \frac{1}{1 + 25x^2}, \qquad -1 \leq x \leq 1$$



Runge Phenomenon: Different Interpolants



Runge Phenomenon: Different Interpolants



- the approximation of f gets worse as n increases;
- the interpolation error is larger near the boundaries of the interval of interest.

Runge Phenomenon

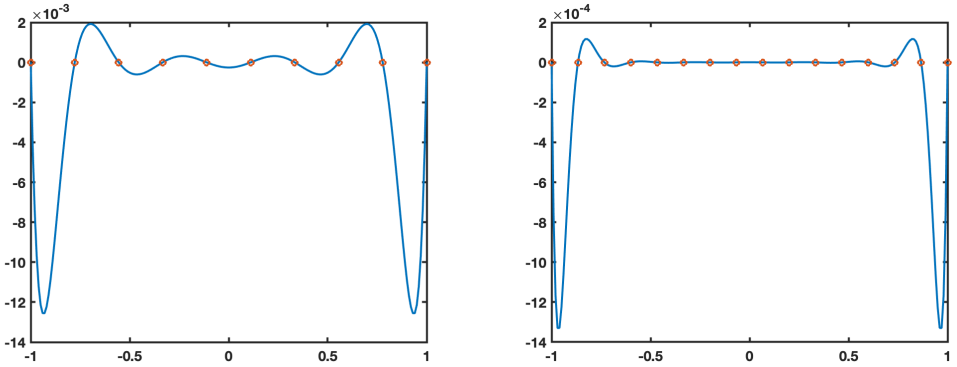
$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)\ldots(x - x_n);$$

$$|e_n(x)| \leq \frac{M}{4(n+1)} h^{n+1};$$

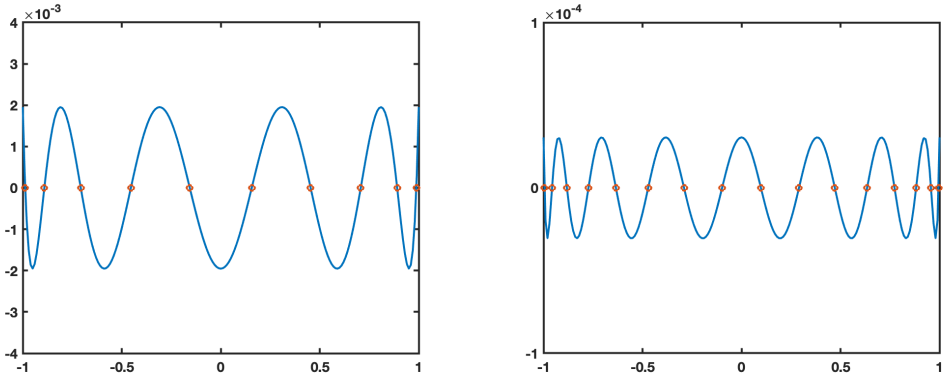
- M is an upper bound of the $n+1$ derivative;
- but the value of the derivative, $f^{(n+1)}(\xi(x))$, changes with x and n;
- for the Runge function, $f^{(n+1)}(x) / (n+1)!$ grows with n near the endpoints of the interval;
- as such, the interpolation error increases with n;
- but the fundamental problem is that we have an *equidistant distribution* of points.

Graph of $\prod_{i=0}^{n}(x - x_i)$

- equally spaced points, $n + 1 = 10, 16$:



- Chebyshev points, $n + 1 = 10, 16$:



Runge Phenomenon: Solution

- do not use an equidistant distribution of nodes;
- it is usually not a good idea to use uniformly spaced interpolation points, unless the degree of the polynomial is low;
- instead, use systems of nodes clustered at the boundary;
- this not a problem of the Lagrange interpolating polys: any other basis will have the same problem since the interpolant is unique;

Chebyshev Points

- choose x_i such that

$$\min_{x_i} \max_{a \leq x \leq b} \prod_{i=0}^{n} (x - x_i);$$

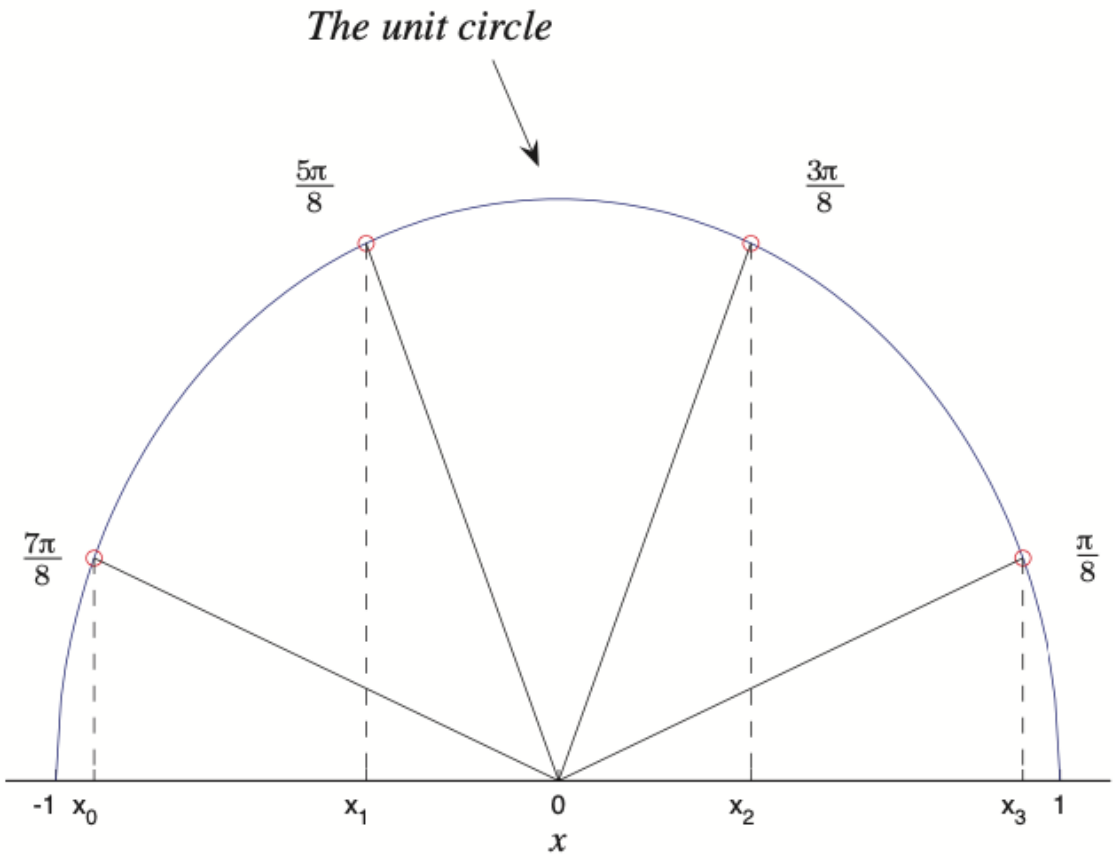
- Chebyshev points have a simple closed form formula in the interval $[-1, 1]$:

$$x_i = \cos\left(\frac{2i+1}{2(n+1)}\pi\right), \quad i = 0, \ldots, n;$$

- and in a general interval $[a, b]$, we scale and translate $[-1, 1]$ onto $[a, b]$

$$x_i \leftarrow a + \frac{b-a}{2}(x_i + 1), \quad i = 0, \ldots, n.$$

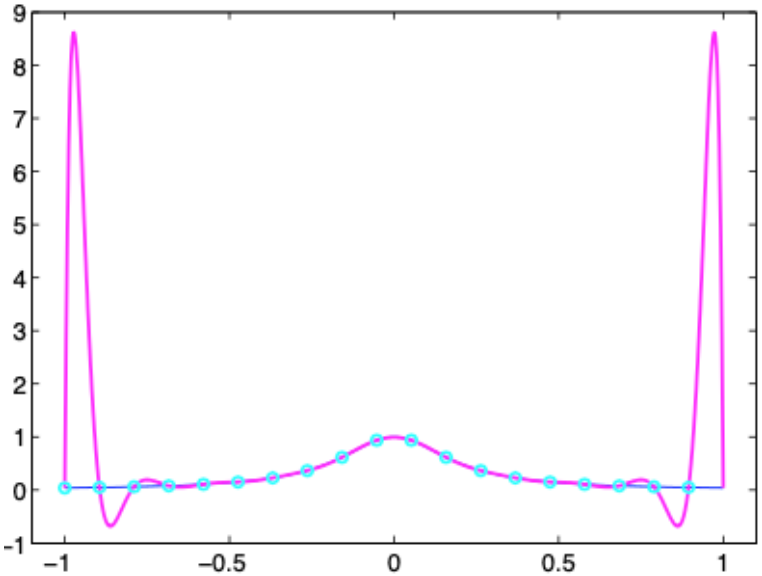
Chebyshev Points

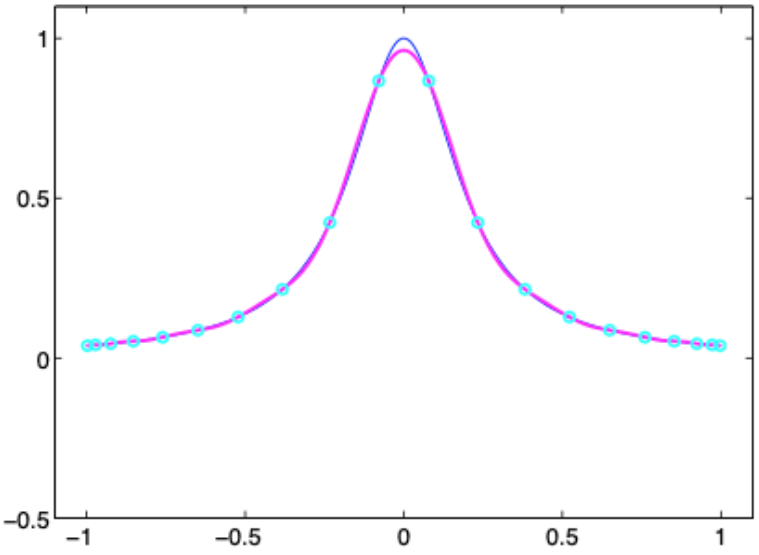


- Chebyshev points cluster at the boundaries
- these nodes are the x-coordinates of equally spaced points on a unit circle.

Runge Function Example

- compare the $n = 20$ interpolant for equidistant and Chebyshev points:





Taylor Approximation vs Interpolant Approximation

- for Taylor approximation:
  1. f can be approximated by a degree n polynomial if we can compute $f(x_0)$ and derivatives of f up to degree n at x_0;
  2. we only need information at a *single point*: x_0;
- for interpolant:
  - the interpolant approximation is also a degree n polynomial, computed from function values at $n+1$ points;
  - needs information from $n+1$ points.